

⚡ ADVANCED COMPILER ENGINEERING

LLVM Kaleidoscope

Compiler Architecture & Implementation

A Comprehensive Deep-Dive into Lexical Analysis, Recursive Descent Parsing, AST Design, Semantic Scoping, Error Handling, LLVM IR Lowering, and Target Architecture

📁 **Repository:** `llvm-kaleidoscope`

⚙️ **Backend:** LLVM IRBuilder & Target Machine

💻 **Language:** C++14 / Modern OOP

🎯 **Topic:** Compiler Frontend, Lowering & Target Strategy

Project Motivation & 3 Core Objectives

01 Frontend Lexing & Parsing

Objective 1 • Grammar Engine

Implement a high-speed, hand-written character tokenizer and hybrid **Recursive Descent / Operator-Precedence Climbing Parser** without relying on third-party generators (Flex/Bison).

- Stateful lexical stream scanning with whitespace/comment stripping
- Accurate mathematical operator precedence resolution (<, +, -, *)
- Grammar support for function prototypes (def) and C-FFI (extern)

02 AST & Semantic Scoping

Objective 2 • AST & Type Safety

Construct a clean, object-oriented **Abstract Syntax Tree (AST)** hierarchy with resilient **Semantic Scoping** and 4-tier error handling.

- Polymorphic class nodes utilizing `std::unique_ptr` for safe memory management
- Function-level symbol table management (NamedValues)
- Rigorous arity checking, unknown variable tracking, and transactional IR rollback

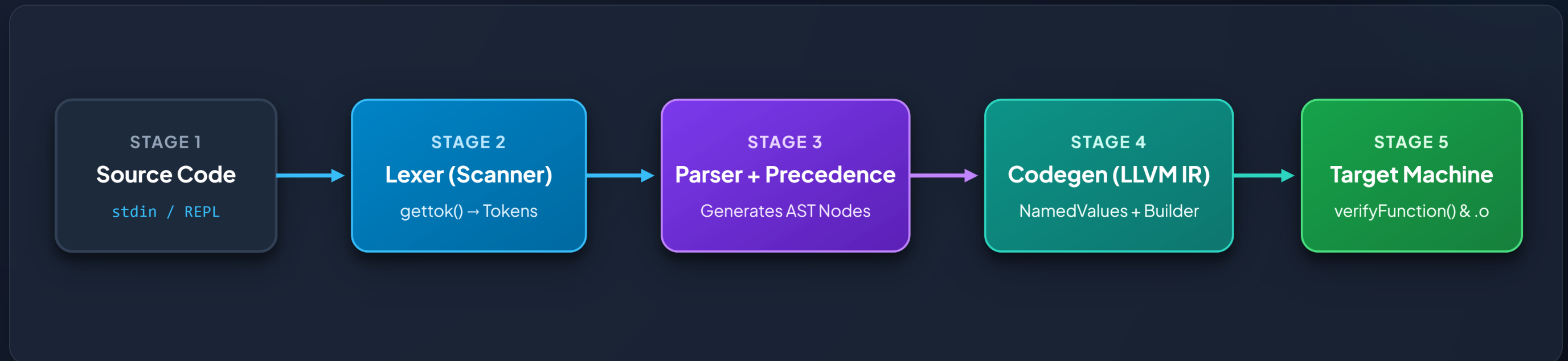
03 LLVM IR & Target Strategy

Objective 3 • Codegen & Target

Lower AST constructs into verified **LLVM SSA Intermediate Representation** and define cross-platform **Target Architecture** lowering for native machine code.

- Direct SSA instruction generation via `llvm::IRBuilder<>`
- Automated function integrity verification via `llvm::verifyFunction`
- Target Machine Triple configuration and native object file (.o / .obj) emission pipeline

High-Level Compiler Pipeline



Interactive REPL Driver

The driver in `main.cpp` operates in a continuous loop, classifying user input into definitions, extern declarations, or immediate top-level expressions wrapped in anonymous functions.

Verifier & Integrity Guard

Every function generated is checked by LLVM's `verifyFunction` pass, catching type mismatches, invalid basic block terminations, or unassigned SSA registers.

Lexical Analysis (Lexer Design)



Scanner Architecture

The lexer is implemented as a stateful character-by-character scanner in `lexer.cpp` driven by `gettok()`.

- **Whitespace Filtration:** Automatically skips spaces, tabs, and linebreaks via `isspace()`.
- **Identifier & Keyword Buffer:** Accumulates alphanumeric characters into `IdentifierStr`. Returns `tok_def`, `tok_extern`, or `tok_identifier`.
- **Numeric Literal Processing:** Matches continuous digit sequences and decimal points into `NumVal` via `strtod()`.
- **Comment Stripping:** Scans from `#` to end-of-line and recursively requests next token.
- **Operator Passthrough:** Returns unknown characters (e.g. `+`, `*`, `()`) as their raw ASCII integer value `[0-255]`.

```
// Token Enumeration in lexer/token.h
enum Token {
    tok_eof = -1,
    // Commands
    tok_def = -2,
    tok_extern = -3,
    // Primary Entities
    tok_identifier = -4,
    tok_number = -5
};

// Scanning Logic in lexer/lexer.cpp
if (isalpha>LastChar)) {
    IdentifierStr = LastChar;
    while (isalnum>LastChar = getchar()))
        IdentifierStr += LastChar;

    if (IdentifierStr == "def") return tok_def;
    if (IdentifierStr == "extern") return tok_extern;
    return tok_identifier;
}
```

Parser Architecture & Operator Precedence

Operator Precedence Parsing

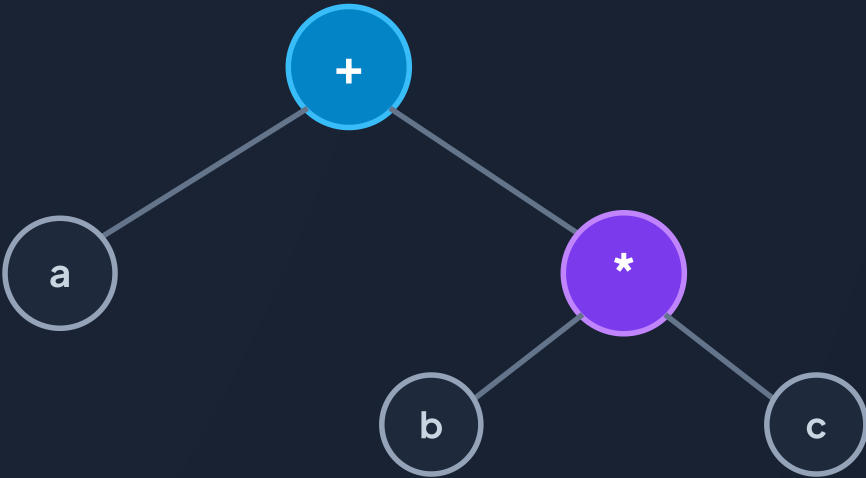
Kaleidoscope utilizes a hybrid **Recursive Descent Parser** for grammar rules combined with an **Operator-Precedence Climbing** algorithm for mathematical binary expressions.

Precedence Hierarchy Table

< : Priority 10 +, - : Priority 20 * : Priority 40

- ParsePrimary() routes tokens to numbers, parenthesized expressions, or variables/function calls.
- ParseBinOpRHS() dynamically groups subtrees based on comparative token precedence.

Parsing: a + b * c Expression Tree



Operator precedence ensures * binds tighter to b and c, forming the subtree before attaching to +.

Abstract Syntax Tree (AST) Class Hierarchy

1 NumberExprAST

Captures numeric constants like 42.0.

```
double Val;
codegen();
```

2 VariableExprAST

References bound arguments (e.g. x).

```
string Name;
codegen();
```

3 BinaryExprAST

Represents operations: LHS op RHS.

```
char Op;
unique_ptr LHS, RHS;
```

4 CallExprAST

Represents invocations: sin(x, y).

```
string Callee;
vector Args;
```

5 PrototypeAST

Captures function metadata: signature name and parameter identifiers list. Emits `llvm::Function*` declarations.

6 FunctionAST

Combines a `PrototypeAST` with a body `ExprAST`. Manages the function entry basic block and populates argument symbol tables.

Semantic Analysis & Robust Error Handling



Symbol Scoping & Type System

- **Function-Scoped Symbol Table:** Managed via `std::map<string, llvm::Value*> NamedValues`. Cleared at the beginning of each function compilation and populated with bound formal arguments.
- **Unified Floating-Point Domain:** All expressions resolve to 64-bit IEEE double (`getDoubleTy`).
- **Boolean Type Promotion:** Comparison operations (`<`) evaluate to 1-bit integers and are zero-extended to floating-point via `CreateUIToFP`.

```
// Argument scope population in FunctionAST::codegen
NamedValues.clear();
for (auto &Arg : TheFunction->args()) {
    NamedValues[Arg.getName().data()] = &Arg;
}
```



4-Tier Error Handling Architecture

- **Tier 1 — Syntax / Lexical Trapping:** `LogError()` prints diagnostic messages to `stderr` and returns `nullptr`, allowing the REPL parser to skip faulty tokens without terminating.
- **Tier 2 — Scope Resolution Trapping:** `LogErrorV("Unknown variable name")` catches undeclared identifiers in expressions and aborts the invalid branch.
- **Tier 3 — Signature & Arity Verification:** Function invocations strictly compare argument counts against registered prototype definitions (`CalleeF->arg_size() != Args.size()`).
- **Tier 4 — Transactional Module Rollback:** If codegen fails mid-body, the compiler executes `TheFunction->eraseFromParent()`, guaranteeing no corrupt partial IR pollutes the active module.

LLVM IR Code Generation Strategy



LLVM Core Infrastructure

- **LLVMContext (TheContext):** Core environment owning type tables, constant pools, and memory arenas.
- **IRBuilder (Builder):** High-level cursor that inserts instructions into basic blocks and performs constant folding.
- **Module (TheModule):** Top-level container holding compiled functions, symbol tables, and external declarations.
- **SSA Form Guarantee:** Generates instructions directly in Static Single Assignment form without mutable stack allocations.

```
// BinaryExprAST::codegen() in ast/BinaryExprAST.cpp
llvm::Value *BinaryExprAST::codegen() {
    llvm::Value *L = LHS->codegen();
    llvm::Value *R = RHS->codegen();
    if (!L || !R) return nullptr;

    switch (Op) {
        case '+': return Builder.CreateFAdd(L, R, "addtmp");
        case '-': return Builder.CreateFSub(L, R, "subtmp");
        case '*': return Builder.CreateFMul(L, R, "multmp");
        case '<':
            L = Builder.CreateFCmpULT(L, R, "cmptmp");
            return Builder.CreateUIToFP(L,
                llvm::Type::getDoubleTy(TheContext), "booltmp");
    }
}
```


Target Architecture & Native Code Generation



1. Target Triple & Registry

Defines the target platform parameters (Architecture-Vendor-OS-Environment):

```
// Example Target Triples
"x86_64-pc-linux-gnu"
"x86_64-w64-windows-msvc"
"aarch64-apple-darwin"
```

- `InitializeAllTargetInfos()`
- `InitializeAllTargets()`
- `InitializeAllTargetMCs()`
- `InitializeAllAsmPrinters()`



2. TargetMachine & Layout

Configures CPU capabilities, ABI conventions, and memory alignments:

- **DataLayout:** Dictates pointer width (32/64-bit), struct alignment, and endianness.
- **CPU & Features:** Targets specific architectures (e.g., `generic`, `haswell`, `m1`).
- **Relocation Model:** Configured for Position Independent Code (`Reloc::PIC_`) for dynamic linking.



3. Object File Emission

Lowers LLVM IR directly into binary object files (`.o` / `.obj`):

```
Legacy::PassManager pass;
auto FileType =
    CGFT_ObjectFile;
TheTargetMachine->
    addPassesToEmitFile(
        pass, dest, nullptr, FileType);
pass.run(*TheModule);
```

Directly linkable via `clang/gcc/ld` into standalone executables.

Execution Walkthrough: Compiling a Function

Target Source Input: `def foo(x y) x + y * 4.0;`

1. Tokens Generated

```
[tok_def] "def"  
[tok_identifier] "foo"  
['(']  
[tok_identifier] "x"  
[tok_identifier] "y"  
[')']  
[tok_identifier] "x"  
['+']  
[tok_identifier] "y"  
['*']  
[tok_number] 4.0  
[';']
```

2. AST Built

```
FunctionAST(  
  Prototype: foo(x, y),  
  Body: BinaryExprAST(  
    Op: '+',  
    LHS: Variable("x"),  
    RHS: BinaryExprAST(  
      Op: '*',  
      LHS: Variable("y"),  
      RHS: Number(4.0)  
    )  
  )  
)
```

3. Emitted LLVM IR

```
define double @foo(double %x, double %y) {  
entry:  
  %multmp = fmul double %y, 4.000000e+00  
  %addtmp = fadd double %x, %multmp  
  ret double %addtmp  
}
```

Functionality Audit & Gap Analysis

Compiler Feature	Implementation Status	Notes & Architectural Details
Lexical Analysis & Tokens	✓ Completed	Numbers, identifiers, def, extern, comments (#), ASCII operators.
Operator Precedence Parser	✓ Completed	Recursive descent climbing for <, +, -, *, calls, parens.
LLVM IR Code Generation	✓ Completed	Generates valid LLVM SSA instructions with <code>verifyFunction</code> integrity checks.
JIT Runtime Execution Engine	✗ Missing	Top-level expressions emit IR as <code>__anon_expr</code> but are not executed in memory.
Control Flow (if/then/else, loops)	⌚ Planned	No conditional basic block branching (Tutorial Chapter 5).
Mutable Variables & Memory	⌚ Planned	No <code>var</code> keyword or memory stack allocations (<code>alloca</code>) (Tutorial Chapter 7).
PassManager Optimizations	⌚ Planned	No constant folding, dead code elimination, or reassociation passes attached.

Future Work Plan & Implementation Milestones

P1 Phase 1: JIT & Optimization

Milestone 1 • Chapter 4

- Integrate **LLVM OrcJIT / LLJIT** to evaluate anonymous expressions immediately.
- Attach `FunctionPassManager` with Instruction Combining, GVN, and Reassociate passes.
- Enable live REPL evaluation.

P2 Phase 2: Control Flow & State

Milestone 2 • Chapters 5–7

- Implement `IfExprAST` with conditional branch phi-nodes.
- Add `ForExprAST` loop construct.
- Add mutable local variables with `stack allca` and LLVM `mem2reg` pass.

P3 Phase 3: Native Target Emission

Milestone 3 • Chapters 8–9

- Configure target machine triple and data layouts for cross-compilation.
- Emit native object code (`.o` / `.obj`) and link to standalone executables.
- Add DWARF / CodeView debug symbols.

 SUMMARY & WRAP-UP

Thank You! Questions & Discussion

Key Project Takeaways:

- Clear separation of concerns between Tokenizer, Grammar Parser, Semantic Analyzer, and LLVM Backend.
- 4-tier error handling architecture ensuring graceful REPL recovery and transactional IR rollbacks.
- Production-grade lowering to verified LLVM IR and clear target architecture mapping for native binary compilation.

 **Reference:** llvm.org/docs/tutorial

 **Repository:** [llvm-kaleidoscope](https://github.com/llvm-kaleidoscope)